

📄 Documentación Técnica: POC Vista 360 Real-Time (Event-Driven Architecture)

1. Visión General de la Solución

Esta Prueba de Concepto (POC) implementa una **Vista 360 de Cliente** en tiempo real. Resuelve el problema de la fragmentación de datos bancarios integrando eventos de múltiples sistemas (FlexCube, iSeries, Thought Machine y SOAP) mediante una arquitectura de streaming.

Utiliza **Apache Flink** como motor de procesamiento distribuido para aplicar una regla de negocio de **Aritmética de Signos** (Ingresos positivos, Deudas negativas) sobre ventanas de tiempo de 5 segundos. El resultado se consolida en una base de datos de lectura rápida (H2) y se expone a través de una **API REST segura y habilitada para CORS**, lista para ser consumida de forma nativa por aplicaciones Frontend (Angular, React, Vue).

2. Prerrequisitos e Instalación de Herramientas

Para ejecutar esta solución en un entorno local (Windows), es necesario instalar el siguiente software:

- **Java Development Kit (JDK) 17 LTS:** Ejecute `java -version` en la consola de PowerShell para validar.
- **Apache Kafka v4.2.0 (KRaft):** Extraiga el archivo binario `.tgz` y mueva la carpeta a la raíz del disco duro, ruta exacta: `C:\apache\kafka_2.13-4.2.0`.
- **Entorno de Desarrollo (IDE):** Se recomienda IntelliJ IDEA (Community o Ultimate) para ejecutar el backend de Spring Boot y el motor de Flink.

3. Configuración de Infraestructura (Kafka Standalone)

Para maximizar el rendimiento en desarrollo y evitar la complejidad de Zookeeper, Kafka se ejecuta en modo **KRaft Combinado** (Broker y Controller en el mismo nodo).

3.1. Archivo de Propiedades (`broker.properties`)

Navegue a `C:\apache\kafka_2.13-4.2.0\config\`, abra el archivo `broker.properties` y reemplace todo su contenido por el siguiente bloque:

```
# Roles combinados para desarrollo local
process.roles=broker,controller
node.id=1
controller.quorum.bootstrap.servers=localhost:9093

# Puertos de escucha (9092 para Spring/Flink, 9093 para control interno)
listeners=PLAINTEXT://localhost:9092,CONTROLLER://localhost:9093
listener.security.protocol.map=CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT
inter.broker.listener.name=PLAINTEXT
advertised.listeners=PLAINTEXT://localhost:9092
controller.listener.names=CONTROLLER

# Configuración de hilos
num.network.threads=3
num.io.threads=8

# Directorio de almacenamiento de datos
log.dirs=C:/apache/kafka_final_logs
num.partitions=1

# Tópicos internos
offsets.topic.replication.factor=1
transaction.state.log.replication.factor=1
transaction.state.log.min.isr=1
```

3.2. Script de Auto-Arranque (`Arrancar_Kafka.ps1`)

Cree un archivo en su escritorio llamado `Arrancar_Kafka.ps1` con el siguiente código. Este script formatea el disco automáticamente si es la primera ejecución y arranca el servidor previniendo corrupciones de datos.

```
$KAFKA_PATH = "C:\apache\kafka_2.13-4.2.0"
$LOG_PATH = "C:\apache\kafka_final_logs"

Write-Host "--- Iniciando Entorno Kafka KRaft (POC Vista 360) ---" -ForegroundColor Cyan
cd $KAFKA_PATH

if (-not (Test-Path "$LOG_PATH\meta.properties")) {
    Write-Host "[!] Formateando almacenamiento por primera vez..." -ForegroundColor Yellow
    .\bin\windows\kafka-storage.bat format -t POC-VISTA-360-OK -c .\config\broker.properties --standalone
}

Write-Host "[+] Arrancando Servidor Kafka en localhost:9092..." -ForegroundColor Green
Write-Host "[!] Para apagar de forma segura, presione Ctrl+C en esta ventana." -ForegroundColor Magenta
.\bin\windows\kafka-server-start.bat .\config\broker.properties
```

4. Arquitectura de Código (Spring Boot & Flink)

El proyecto está estructurado en capas para separar la ingesta de eventos, el procesamiento distribuido y la exposición de datos al Frontend. Las clases principales son:

- **PocAvcProducerApplication.java**: Clase principal que levanta el servidor Tomcat embebido en el puerto `8080`.
- **FlinkStreamingJob.java**: Motor de procesamiento. Se conecta a los tópicos de Kafka (`tx-flexcube`, `tx-iseries`, etc.), agrupa los eventos en ventanas de 5 segundos y realiza la suma de saldos.
- **TestController.java**: Controlador REST que actúa como productor de eventos hacia Kafka y como consumidor de la Vista 360 para el Frontend.
- **SecurityConfig.java**: Gestiona los filtros de seguridad corporativa y las políticas de CORS.

5. Integración con el Frontend (Contratos API REST)

Para garantizar que las aplicaciones cliente puedan consumir la Vista 360 sin bloqueos de seguridad cruzada del navegador, el backend expone una API REST estandarizada. El endpoint `GET /api/test/resumen/{idCliente}` está diseñado para devolver el estado HTTP correcto (`200 OK` si hay datos, o `404 Not Found` si la base de datos está limpia o el cliente no existe).

Ejemplo de consumo desde el Frontend (JavaScript/Fetch):

```
fetch('http://localhost:8080/api/test/resumen/777')
  .then(response => {
    if (!response.ok) throw new Error('Cliente sin productos o no encontrado (404)');
    return response.json();
  })
  .then(data => console.log('Saldo Global Consolidado:', data.saldoTotalGlobal))
  .catch(error => console.error(error));
```

6. Guía de Ejecución, Pruebas y Simulación de Sistemas Core

Esta sección detalla cómo validar el ciclo de vida completo de la arquitectura (Ingesta -> Mensajería -> Procesamiento -> Vista Materializada).

6.1. Preparación del Entorno

Antes de iniciar la simulación, levante la infraestructura en este orden estricto:

1. **Kafka**: Ejecute el script `Arrancar_Kafka.ps1` en PowerShell.
2. **Flink**: Ejecute la clase `FlinkStreamingJob.java` desde IntelliJ IDEA.
3. **Spring Boot**: Ejecute la clase `PocAvcProducerApplication.java` desde IntelliJ IDEA.

Nota: Al reiniciar el proyecto, la base de datos en memoria (H2) arranca vacía. Si consulta la API REST en este momento, devolverá un código HTTP `404 Not Found`.

6.2. Simulación de Fuentes de Verdad (Inyección de Eventos)

Abra una nueva terminal de **PowerShell** y copie y pegue los siguientes comandos `curl.exe` uno por uno.

1. Ingreso de Capital (Sistema FlexCube - Ahorros):

```
curl.exe -X POST "http://localhost:8080/api/test/flexcube?idCliente=777&saldo=7000000"
```

(Respuesta esperada: Evento FlexCube enviado: 7000000)

2. Consumo de Tarjeta (Sistema Thought Machine - Core Cloud):

```
curl.exe -X POST "http://localhost:8080/api/test/thoughtmachine?idCliente=777&monto=-1200000&tarjetaId=VISA-777"
```

(Respuesta esperada: Evento Thought Machine enviado: -1200000)

3. Vinculación de Póliza (Sistema SOAP Externo - Seguros):

```
curl.exe -X POST "http://localhost:8080/api/test/soap?idCliente=777&monto=800000&poliza=POL-2026"
```

(Respuesta esperada: Evento SOAP enviado: 800000)

4. Cargo por Cuota (Sistema iSeries Legacy - Préstamos):

```
curl.exe -X POST "http://localhost:8080/api/test/iseries?idCliente=777&monto=-3000000"
```

(Respuesta esperada: Evento iSeries enviado: -3000000)

6.3. Verificación de la Aritmética de Signos (Motor Flink)

Apache Flink captura los 4 eventos en su ventana de 5 segundos y ejecuta el cálculo consolidado en memoria:

- 7.000.000 (FlexCube) - 1.200.000 (Thought Machine) + 800.000 (SOAP) - 3.000.000 (iSeries) = 3.600.000

6.4. Validación de la API REST (Vista del Frontend)

Para confirmar el resultado de la integración, abra un navegador web o Postman, y realice una petición GET a la ruta: ↗

http://localhost:8080/api/test/resumen/777

Comportamiento Esperado: La API responderá con un código 200 OK y el siguiente cuerpo JSON exacto:

```
{
  "idCliente": "777",
  "nombreCliente": "Cristian Munoz",
  "saldoTotalGlobal": 3600000.0,
  "productos": [
    {
      "tipo": "Cuenta Ahorros",
      "sistema": "FlexCube",
      "identificador": "AHO-777",
      "estado": "Activa",
      "valor": 7000000.0
    },
    {
      "tipo": "Préstamo Personal",
      "sistema": "iSeries",
      "identificador": "PREST-777",
      "estado": "MORA",
      "valor": 3000000.0
    },
    {
      "tipo": "Tarjeta de Crédito",
      "sistema": "Thought Machine",
      "identificador": "TDC-777",
      "estado": "Activa",
      "valor": 1200000.0
    },
    {
      "tipo": "Seguro de Vida",
      "sistema": "SOAP",
      "identificador": "POL-777",
      "estado": "Vigente",
      "valor": 800000.0
    }
  ]
}
```

7. Procedimiento de Apagado Seguro

Para evitar la retención de puertos o corrupción en el almacenamiento de Kafka:

1. Detenga la ejecución de Spring Boot y Flink en IntelliJ IDEA usando el botón Stop.
2. En la terminal de PowerShell donde corre Kafka, presione **Ctrl + C**, confirme con **S** o **Y** si se solicita, y espere a que el proceso libere la consola por completo.